As PHP programmers, we don't need to worry too much about any of this low-level stuff (maybe buy a drink for the next assembly programmer you encounter), but it's good to know what's going on and why.

Pitfalls of Automatic Conversion

Hopefully the previous few examples have demonstrated how much is really happening when types are converted. PHP's automatic juggling of types is certainly convenient: we've just seen how we can treat numerical strings as numbers, or numbers as strings, and we didn't need to do anything special in our code to make that happen. There are also mechanisms for explicitly converting types (we'll see some in a bit), which trigger the same data changes.

This is great when it works, but there are times when it either doesn't work at all (usually emitting an error) or, worse, there are times when conversion is successful but with unintended results. Let's look at a few examples.

Casting composite types to scalar usually results in some form of error and possible termination of the program. Here's what happens when we try to use an array as a string or an integer:

```
var_dump("This is what happens when we stringify an array: " . ['an',
'array']);
var_dump(['an', 'array'] + 1);
```

```
Output:
Warning: Array to string conversion
string(54) "This is what happens when we stringify an array: Array"
Fatal error: Uncaught TypeError: Unsupported operand types: array + int
```

Using an array in a string context gives us a warning, but we at least get a usable (but not necessarily useful) string as the result, and our code continues to run. The mathematical operation that followed had an unhappier outcome: a fatal TypeError was emitted.